1. Introduction and Goals

1.1 Requirements Overview

The WeatherArchive frontend fulfills the user facing backlog from the Semester Project Specification and the evaluation matrix:

- **Search & Navigation (Stories 1–5)**: landing page with logo, guided search bar, autocomplete restricted to curated cities, and routing to the city detail page.
- **Observation Experience (Stories 6–10)**: detail page combines hero media slot for images, interactive chart, date picker, hour slider, and contextual error handling.
- **Data Ingestion & API (Stories 11–30)**: currently scoped as future backend services (webcam ingestion, API security, storage, video creation). They are documented as planned interfaces and constraints.
- **Non-functional Deliverables (Stories 31–35)**: arc42 documentation, presentation, frontend draft, and code quality practices captured in this document and the repository tooling.

1.2 Quality Goals

Priority	Goal	Description
Q1	Fast perception	Search and navigation respond in <300 ms locally; image loads provide feedback so users perceive responsiveness even when assets are missing.
Q2	Reliable data access	Date/hour controls never allow unavailable combinations; optimistic UI always communicates when data or media cannot be displayed.
Q3	Deployable artifact	`npm run build` must produce a deterministic static bundle that can be synced to S3/CloudFront without manual tweaks.
Q4	Evolvable backend	Frontend isolates backend calls behind fetch utilities so upcoming API increments (stories 11–30) can be integrated without refactors.

1.3 Stakeholders

Role/Name	Contact	Expectations
Product Owner /	Semester project	Traceability to specification, measurable quality

Role/Name	Contact	Expectations
Lecturer	supervisor	goals, presentable demo.
End Users (Students, Public)	Via web UI	Straightforward search, meaningful media + metrics per city/time.
Webcam Providers	API client backlog	Secure upload endpoints, API keys, predictable error codes.
DevOps / Cloud Team	AWS contact	Clear S3/CloudFront deployment runbook, secrets separation, minimal manual steps.

2. Architecture Constraints

- **Technology**: SvelteKit + Vite, Tailwind CSS, Chart.js, TypeScript per specification.
- **Tooling**: Node.js 20+, npm, ESLint/Prettier enforced by CI instructions.
- **Deployment**: Static assets uploaded to an AWS S3 bucket (optionally fronted by CloudFront) instead of the former NGINX container image.
- **Assets**: Weather images stored under `static/images` with naming convention `{cityslug}_{YYYY-MM-DD}_{HH}.jpg` to satisfy evaluation matrix story 6.
- **Security**: Future API endpoints must require API keys (stories 12, 13) and integrate with AWS credential handling.

3. Context and Scope

3.1 Business Context

Actor	Interaction	Notes
Viewer	Uses browser UI to search cities and inspect historical weather slices.	Inspired by Google-like UX (stories 1–5).
Webcam Device	Planned to POST images + metadata to backend.	Awaiting API implementation (stories 11–15).
API / Data Services	Serve curated weather data, metadata, and media references.	Interfaces defined in spec; currently mocked in frontend.

3.2 Technical Context

External System	Technology	Interface
Browser	HTML5, JS, CSS	SvelteKit routes, static assets, fetch calls.

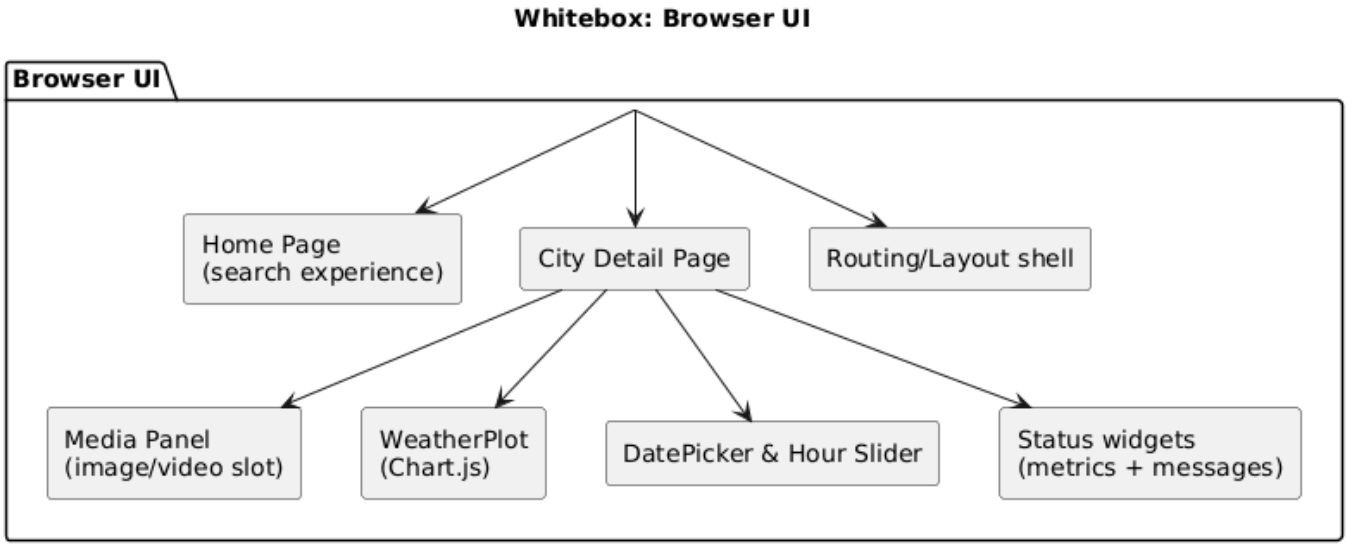
External System	Technology	Interface
Backend API (planned)	REST/JSON over HTTPS, Lambda Functions	`/api/events`, `/api/weather` (per spec).
Object Storage	AWS S3 (planned)	Stores images/videos, accessed via signed URLs.
Redis Cache	AWS ElastiCache (planned)	Reduces database hits for repeated queries.
Database	RDS Postgres	Stores meta data

4. Solution Strategy

1. **Frontend-first**: deliver all user-visible stories with static mocks to satisfy evaluation checkpoints while backend matures.
2. **Deterministic builds**: Vite build output stays static so `aws s3 sync build/ s3://bucket` reproduces the site identically on each deployment.
3. **Progressive enhancement**: UI controls work offline with mock data but expose clear extension points for live API integration.
4. **Asset fallback**: Image loader displays skeletons and descriptive failure panels when files are absent, aligning with quality goal Q2.

5. Building Block View

5.1 Whitebox Overall System



- **Motivation**: Keep frontend modular so each user story maps to a component (search, detail, error handling) and future backend calls are centralized.
- **Contained Building Blocks**:

    - `src/lib/pages/HomePage.svelte`: autocomplete search, navigation logic.
    - `src/lib/pages/CityDetailPage.svelte`: orchestrates image selection, weather widgets.
    - `src/lib/components/*`: reusable UI controls (Header, DatePicker, WeatherPlot, SearchBox, ErrorMessage).
    - `static/images/*`: weather media catalog.
- **Important Interfaces**: Component props/events, SvelteKit routing (`/` , `/cities/[city]`), fetch adapters (future work).

5.1.1 Home Page Module

- **Responsibility**: Provide search UX, filter curated capitals, navigate to detail route.
- **Interfaces**: Emits `select` events to `goto()` , consumes static capital list.
- **Location**: `src/lib/pages/HomePage.svelte`.
- **Fulfilled Stories**: 1–5.

5.1.2 City Detail Module

- **Responsibility**: Combine media viewer, chart, date/hour controls, and error states.
- **Interfaces**: Binds to DatePicker events, slider input, image load events.
- **Location**: `src/lib/pages/CityDetailPage.svelte`.
- **Fulfilled Stories**: 6–10.
- **Open Issues**: Needs live data provider (stories 11–30).

5.1.3 WeatherPlot Component

- **Responsibility**: Render multi-axis chart for temperature/humidity/pressure.
- **Interfaces**: Chart.js config with placeholder data.
- **Location**: `src/lib/components/WeatherPlot.svelte`.
- **Fulfilled Stories**: Part of story 6.
- **Risks**: Replace mock data once backend endpoints stabilize.

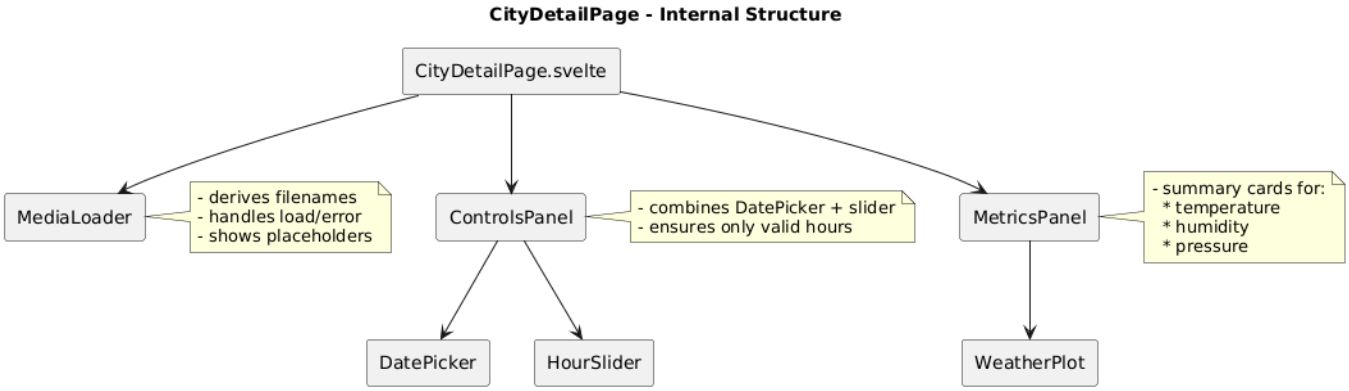
5.1.4 DatePicker Component

- **Responsibility**: Display selectable dates with availability constraints.
- **Interfaces**: `availableDates` , `minDate` , `maxDate` , emits `change` .
- **Location**: `src/lib/components/DatePicker.svelte`.
- **Fulfilled Stories**: 7, 8.

5.2 Level 2

Detailed structure of City Detail Page:

- **MediaLoader**: encapsulated logic in `CityDetailPage.svelte` for deriving filenames, handling load/error, and showing placeholders.
- **ControlsPanel**: `DatePicker` + slider combination ensures only valid hours.
- **MetricsPanel**: summary cards for temperature, humidity, pressure (currently derived from mock object).



5.3 Level 3

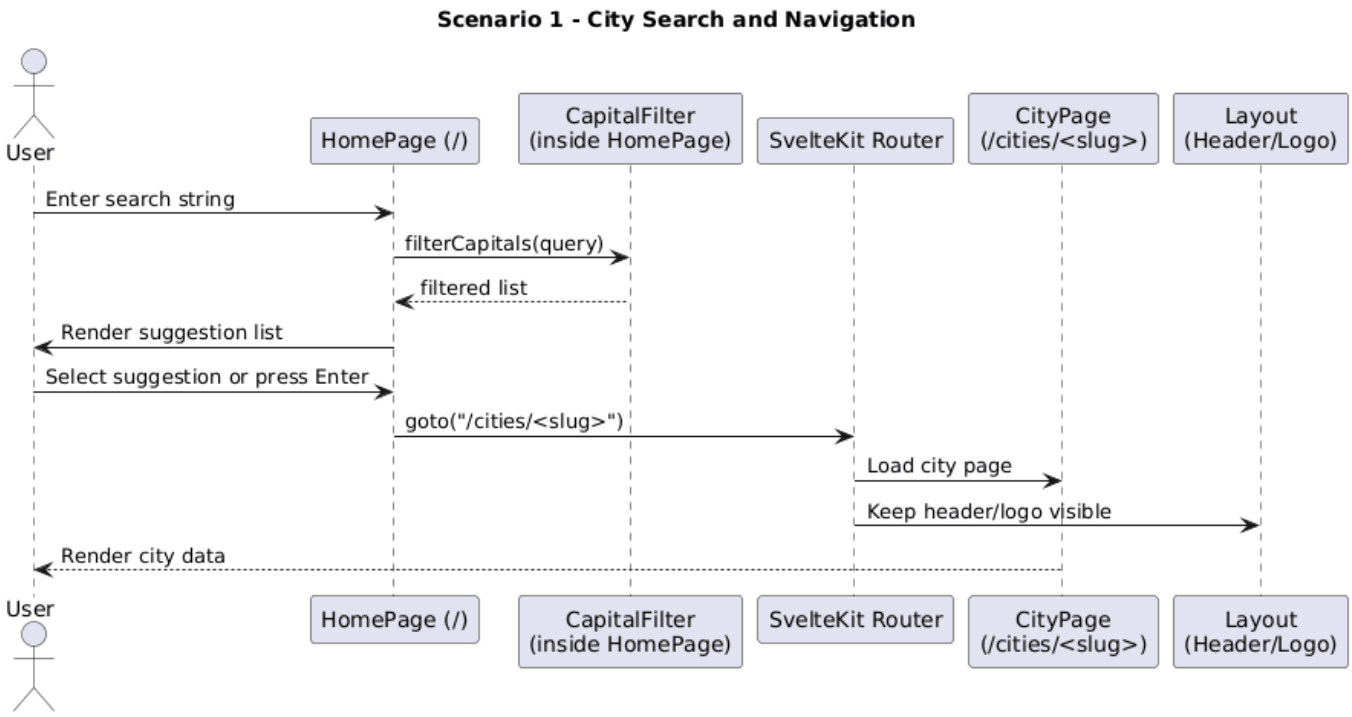
Lower-level elements (e.g., slider, error card) rely on native HTML inputs and Tailwind classes.

6. Runtime View

- TODO - Sequence-Diagrams
- TODO - Question: Do we need more scenarios? (eg. lambda calls, webcam api interactions...)

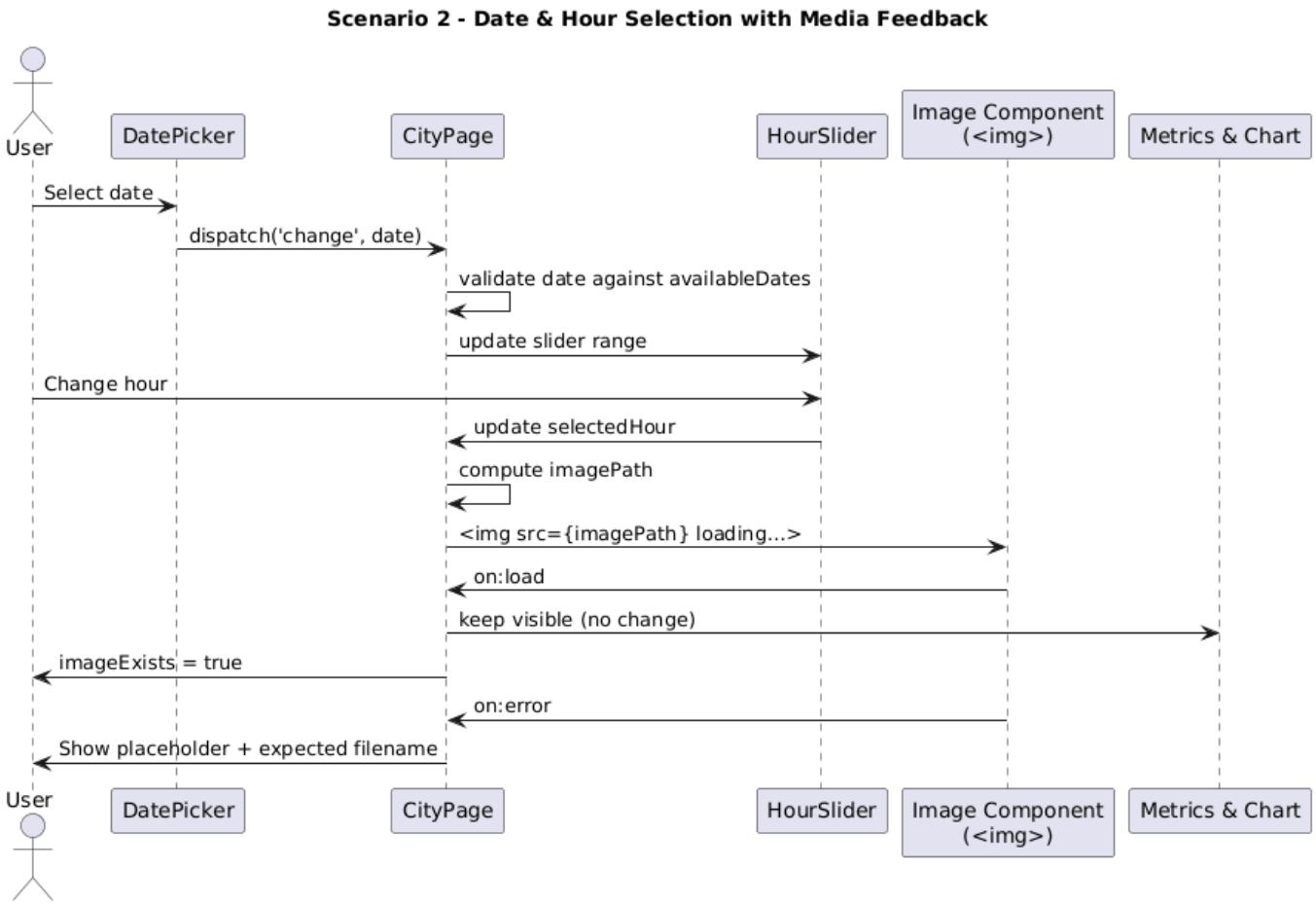
6.1 Scenario 1 – City Search and Navigation

1. User enters a string on `/`.
2. `HomePage` filters capitals and renders the suggestion list.
3. On selection or enter, SvelteKit `goto` pushes `/cities/<slug>`.
4. Layout component keeps header/logo visible for navigation (story 9).



6.2 Scenario 2 – Date & Hour Selection with Media Feedback

1. User selects a date via `DatePicker`; component dispatches `change` event.
2. City page validates date against `availableDates` and updates the bound slider range.
3. When hour changes, the computed `imagePath` updates; `<img>` begins loading.
4. `on:load` toggles `imageExists`; metrics and chart remain visible.
5. `on:error` surfaces descriptive placeholder plus expected filename, fulfilling story 10.



7. Deployment View

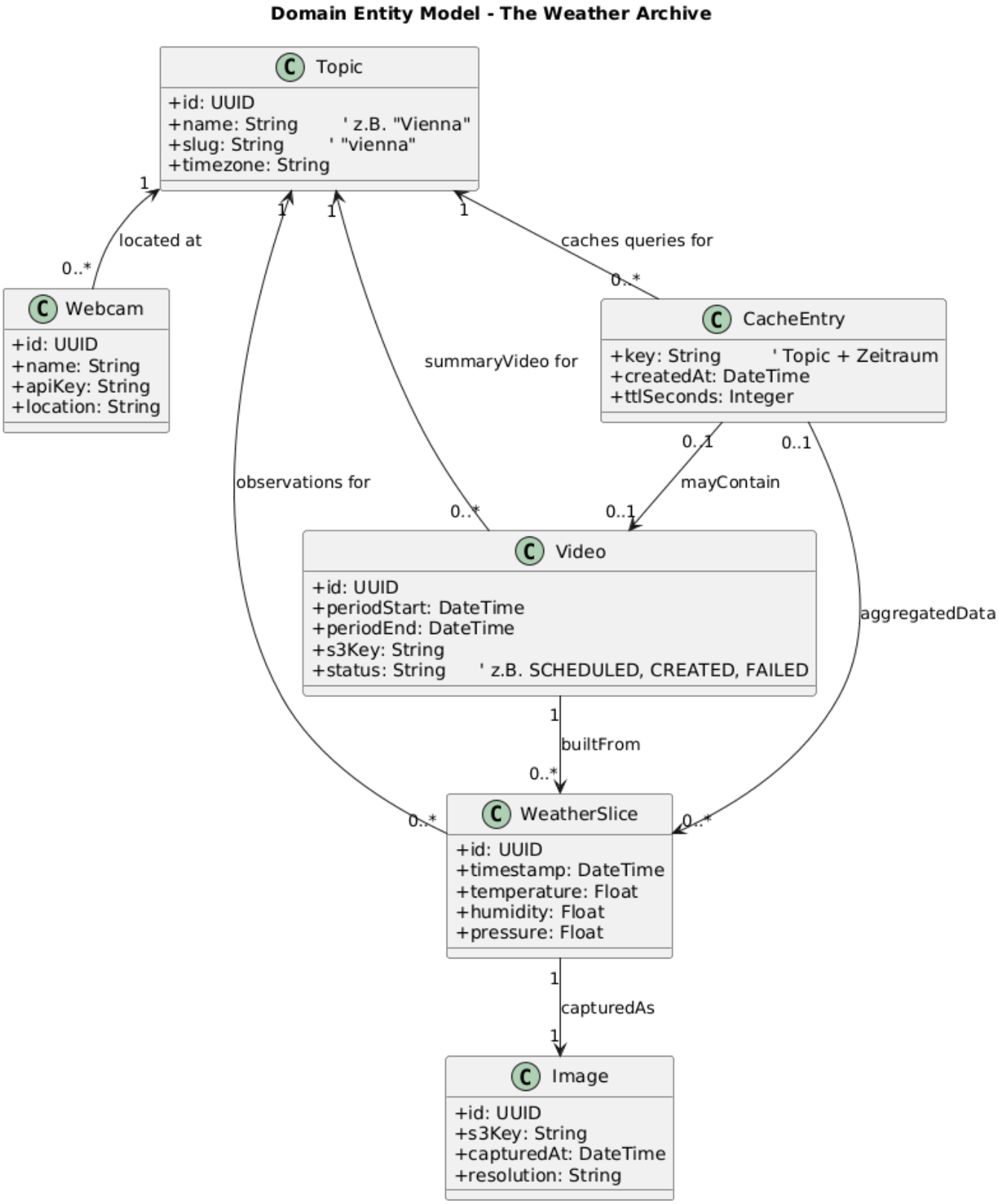
7.1 Infrastructure Level 1

- **Motivation**: Provide reproducible deployments for grading and AWS hosting.
- **Quality Attributes**: Immutable artifacts, quick propagation through AWS edge caches, minimal ops steps.
- **Mapping**: `npm run build` creates the static `build/` folder; the output is uploaded to an AWS S3 bucket (e.g., `weatherarchive-prod`) and optionally distributed via CloudFront. No container runtime is required.

7.2 Infrastructure Level 2

- **Local Dev**: `npm run dev` with Vite's hot module reload.
- **Artifact Publishing**: `npm run build` followed by `aws s3 sync build/ s3://<bucket-name>/ --delete` (or CI equivalent) to update the website. (PLANNED)
- **AWS Hosting**: S3 static website hosting serves the SPA (Single Page Application), CloudFront handles HTTPS/CDN, and future APIs live behind API Gateway + Lambda + RDS/Redis per spec.

8. Cross-cutting Concepts



- **Styling & Branding**: Tailwind design tokens (`app.css`) maintain consistent typography and colors for both pages.
- **Error Handling**: `ErrorMessage.svelte` centralizes readable messages per story 10.

- **Configuration**: Available dates/hours currently hardcoded but isolated so they can be fetched; image naming enforced globally.
- **Observability**: Runtime feedback (loading spinners, placeholders) proxies for traditional logging in the static frontend; future API integration should forward correlation IDs.
- **Security (planned)**: API key headers, HTTPS endpoints, and AWS IAM roles as mandated by stories 12–22 once backend components exist.

9. Architecture Decisions

ID	Decision	Status	Rationale
ADR-01	Use SvelteKit + Tailwind for UI	Accepted	Matches course tech stack, supports fast prototyping.
ADR-02	Host static build on AWS S3 (+ optional CloudFront)	Accepted	Keeps deployment simple and integrates with AWS-native services.
ADR-03	Mock data until backend ready	Accepted w/ debt	Keeps focus on UX backlog while API stories are pending.
ADR-04	Image naming convention `{city}_{date}_{hour}`	Accepted	Enables deterministic lookup without metadata service.

10. Quality Requirements

10.1 Quality Requirements Overview

Attribute	Description
Usability	Users immediately recognize logo/search entry points; controls show availability hints.
Performance	Initial page load budget < 1.5 MB (current static bundle fits); interaction latency dominated by local filtering.
Reliability	Deterministic filenames prevent 404 ambiguity; app surfaces guidance when images are missing.
Maintainability	Components stay isolated so new sensors or metrics can be added without touching the routing shell.

10.2 Quality Scenarios

Scenario	Trigger	System Reaction
Image Missing	Expected media file not present in `static/images` or S3	Placeholder panel displays filename + guidance so the user understands the gap.
Invalid Date Selection	User selects date/hour without data	DatePicker + slider restrict values; `ErrorMessage` tells the user to choose an available slot.
Deployment Regression	`npm run build` fails or emits warnings	CI/deployer halts; no files uploaded to S3 until build succeeds, preventing half-baked releases.

11. Risks and Technical Debts

- **Backend Gap**: Stories 11–30 remain unimplemented; frontend mocks need replacement before live demo with webcams.
- **Asset Scale**: Up to 14 images per city/day currently planned; growth requires CDN/S3 integration to avoid bloated repository.
- **Caching Strategy**: Redis stories (18–22) are unaddressed; document placeholders but plan iteration.
- **Video Service**: Future addition of scheduled video rendering (stories 26–30) may impact UI (player component) and storage cost.

12. Glossary

Term	Definition
Weather Slice	Combination of city, date, and hour representing one observation.
Capital Catalog	Curated list of European capitals offered in autocomplete.
Media Slot	UI area currently showing an image but designed to host video playback later.
Evaluation Matrix	Spreadsheet in `docs/Evaluation_matrix_for_the_semester_project.md` enumerating backlog items and status.
Semester Project Specification	Official PDF outlining functional and non-functional requirements for the course project.